# Trabalho II de Programação Orientada Objetos

## Sistema de Clinica Veterinaria

Instituição: Universidade Estadual do Centro Oeste

Curso: Ciência Da Computação
Davi Belo Tussolini

1.Diagnóstico breve do projeto original.

O projeto original já possuía uma estrutura de aplicação em Java com acesso a banco de dados por meios de classes DAO. Entretanto a organização inicial concentrava a criação das implementações de acesso a dados somente numa fábrica e não apresentava de forma explícita uma camada própria para centralizar as regras.

Com isso, a aplicação foi adaptada para o contexto de uma clínica veterinária, contemplando as entidades Tutor, Animal, Veterinário. A tabela Consulta ainda não foi implementada na FXML no qual pretendo.

2.Padrões aplicados

Foram considerados três elementos do projeto: o padrão arquitetural Service Layer e os três padrões GoF Factory Method, Decorator e Builder.

2.1 Service Layer

Identificação: Finalidade centralizar as regras de negócio e as validações da aplicação, mantendo os Controles separados do acesso direto ao Banco de dados.

Classes e pacotes envolvidos: Pacote Service, AnimalService,TutorService, VeterinarioService, Interfaces Dao.

Situação Anterior: As operações de cadastro, atualizaçãoes e exclusão precisavam de validações e regras de negócios, Sem uma camada de serviços bem definida, essas regras poderiam ficar distribuídas entre controllers e DAOs, dificultando a manutenção e reutilização

Solução Implementada: Classes de serviços criadas recebem os dados da aplicação, validam as informações e, quando os dados são válidos, delegam a persistências ao DAO correspondente

```
private void validarAnimal(Animal animal) {
    if (animal == null) {
        throw new
IllegalArgumentException("Animal não
informado.");
    }

    if (animal.getNome() == null ||
animal.getNome().isBlank()) {
        throw new IllegalArgumentException("O
nome do animal é obrigatório.");
    }

    if (animal.getEspecie() == null ||
animal.getEspecie().isBlank()) {
```

```
        throw new IllegalArgumentException("A
espécie é obrigatória.");
    }

    if (animal.getTutorID() <= 0) {
        throw new
IllegalArgumentException("Selecione um
tutor.");
    }

    if (animal.getDataNascimento() != null &&

animal.getDataNascimento().isAfter(LocalDate.no
w())) {
        throw new IllegalArgumentException("A
data de nascimento não pode ser futura.");
    }

    if (animal.getSexo() != null &&
            !animal.getSexo().equals("Macho") &&
            !animal.getSexo().equals("Fêmea")) {
        throw new
IllegalArgumentException("Selecione um sexo
válido.");
    }

    animal.setNome(animal.getNome().trim());

animal.setEspecie(animal.getEspecie().trim());

animal.setRaca(normalizar(animal.getRaca()));
}
```

Benefícios Esperado: centralizar as regras de negócio, evitar duplicação de validações e facilitar alterações futuras sem misturar lógica de interface com persistência.

Forma de verificação**:** a implementação é utilizada quando os Controllers chamam os métodos dos Services. Pode ser verificada tentando cadastrar um animal sem nome ou sem tutor o Service deve impedir a operação e apresentar a mensagem de validação. Também podem ser testados cadastro, atualização e exclusão com dados válidos.

2.2 Factory Method

Identificação: Finalidade encapsular a criação das implementações DAO, permitindo que cada fábrica concreta determine qual objeto de acesso a dados será instanciado

Pacotes Envolvidos: Pacote Factory, DAOFactory, PostgresAnimalDAOFactory PostgresTutorDAOFactory PostgresVeterinarioDAOFactory

Situação Anterior: Na estrutura anterior, uma única fábrica concreta disponibiliza métodos para criar vários tipos de DAO. Essa solução centralizava a criação.

Solução implementada: A estrutura foi reorganizada para que DAOFactory declare o método abstrato criarDAO(). Cada fábrica concreta estende DAOFactory e implementa esse método, retornando sua implementação específica. A fábrica de animais também envolve AnimalDBDAO com AnimalDAODecorator.

```java
public abstract class DAOFactory<T> {

    public abstract T criarDAO();

}
```

```java
public class PostgresAnimalDAOFactory extends
DAOFactory<AnimalDAO> {

    @Override
    public AnimalDAO criarDAO() {
        return new AnimalDAODecorator(new
AnimalDBDAO());
    }
}
```

Benefício esperado: separar a decisão de instanciação do código que utiliza os DAOs e permitir trocar a implementação concreta sem alterar as operações de negócio.

Forma de verificação: a implementação é utilizada nos construtores dos Services, que obtêm os DAOs por meio de criarDAO(). A verificação pode ser feita executando operações de cadastro e consulta e confirmando que cada Service utiliza a implementação PostgreSQL correspondente.

## 2.3 Decorator

Identificação: Finalidade**:** acrescentar comportamento às operações de AnimalDAO sem modificar a implementação original de acesso ao banco de dados.

Classes e pacotes envolvidos**:** Pacote decorator; AnimalDAODecorator Interface AnimalDAO Implementação AnimalDBDAO

Situação anterior: As operações de acesso aos animais precisavam fornecer informações de acompanhamento, como mensagens de registro das ações. Alterar diretamente AnimalDBDAO para adicionar esse comportamento misturaria responsabilidades de persistência e registro.

Solucão implementada: Foi criada AnimalDAODecorator, que implementa AnimalDAO construtor e delega as operações ao objeto recebido. Antes e depois das chamadas , registra mensagens de LOG no console.

Papeis das Classes: AnimalDAO define a interface comum. AnimalDBDAO executa a persistência. AnimalDAODecorator envolve outro AnimalDAO e adiciona o registro das operações.

```
public class AnimalDAODecorator implements
AnimalDAO {


   private final AnimalDAO dao;


   public AnimalDAODecorator(AnimalDAO dao) {
       this.dao = dao;
```

```
    }


    @Override
    public void insere(Animal animal) throws
SQLException {
        System.out.println("[LOG] Cadastrando
animal: " + animal.getNome());


        dao.insere(animal);


        System.out.println("[LOG] Animal
cadastrado com sucesso.");
    }
```

Benefício esperado: adicionar funcionalidades de acompanhamento sem alterar a classe DAO original e mantendo a possibilidade de utilizar qualquer implementação compatível com AnimalDAO.

Forma de verificação: a fábrica de animais instancia AnimalDAODecorator envolvendo AnimalDBDAO.

Ao cadastrar, atualizar, excluir ou consultar animais pelo fluxo normal do sistema, as mensagens correspondentes devem aparecer no console, enquanto a operação continua sendo executada no PostgreSQL.

2.4 Builder

identificação: Finalidade facilitar a criação de objetos Animal por etapas, com métodos encadeados e validações antes da construção do objeto final.

Classes e pacotes envolvidos: Pacote model AnimalBuilder

Situação anterior: A criação de um Animal exige preenchimento de diferentes atributos, alguns obrigatórios e outros opcionais. Construir o objeto diretamente pode deixar a inicialização menos clara e espalhar verificações pelo código.

Solução implementada: Foi criada a classe AnimalBuilder, que mantém os valores dos atributos e oferece métodos encadeados, como nome(), especie(), raca(), dataNascimento(), sexo() e tutorID().

O método build() verifica os campos obrigatórios, cria Animal e transfere os valores para o objeto. AnimalBuilder organiza a construção e valida os dados mínimos. Animal representa o objeto final utilizado pelo restante da aplicação.

```
Animal animal = new Animal();

animal.setNome(nome);

animal.setEspecie(especie);

animal.setRaca(raca);

animal.setDataNascimento(dataNascimento);

animal.setSexo(sexo);

animal.setTutorID(tutorID);
```

```
animal.setVeterinarioID(veterinarioID);
```

Benefício esperado: tornar a criação de animais mais legível, permitir definir somente os atributos necessários em cada situação e concentrar as verificações iniciais no processo de construção.

Forma de verificação: a verificação consiste em utilizar o Builder para construir um A nimal com dados válidos e confirmar que os atributos foram atribuídos corretamente.